AutoIA: From Mathematical Theory to Source Code

Implementation Map Through Version 0.21.0.dev1

August 2026

Abstract

This document connects the mathematical specification of AutoIA to the concrete Python implementation in version 0.21.0.dev1. It is intended for a researcher or developer who understands the enclosure theory and wants to know exactly where that theory is realized in the source tree. The organizing principle is not a line-by-line code listing. Instead, each mathematical invariant, theorem, or algorithm is mapped to the production module that proposes it, the module that certifies it, the representation object that carries it, and the regression or audit tests that protect it.

A recurring distinction is made between *proposal code* and *proof code*. Proposal code is allowed to use ordinary NumPy floating-point operations because a later proof validates the stored proposal. Proof code is either in the trusted binary64 layer or is a composition of trusted interval operations. This document highlights that boundary explicitly so future changes do not accidentally move an unverified calculation into the trusted path.

Contents

1 Repository architecture at version 0.21.0.dev1 **3**

2 One-box call path from public API to proof result **3**

3 Public input intervals and rigorous centering **4**

4 Representation-independent value dispatch **5**

5 Polynomial algebra: equations to functions **5**
5.1 Multi-indices 5
5.2 Monomial magnitude 6
5.3 Addition 6
5.4 Truncated multiplication 6
5.5 Reciprocal proposal 6

6 Polynomial range methods in code **6**

7 Trusted binary64 layer: the proof boundary **7**
7.1 Long dot products 7

8 Elementary functions: trusted scalar kernel plus generic Taylor lifting **8**

9 Power semantics in code **8**

10 Local NumPy/math/SciPy semantic dispatch **8**
10.1 Prepared shadow function 8
10.2 NumPy proxy 8
10.3 SciPy compatibility 9

11 Semantic linear solve: theory-to-code map **9**
11.1 General polynomial predictor 9
11.2 Residual coefficient construction 9
11.3 Preconditioner portfolio 9
11.4 Defect and contraction proof 9
11.5 Generator-aware solve 10
11.6 Dynamic solve context 10

12 Path-aware control flow in code **10**

13 Adaptive subdivision and cheap split scoring **10**

14 Prepared sessions and certificate restriction in code **11**
14.1 Public preparation object 11
14.2 Exact cache 11
14.3 Containment cache 11
14.4 Restriction function 11
14.5 Interval representation restriction 12

15 Many-box CPU execution in code **12**

16 Result objects and mathematical diagnostics **12**

17 Typed failure semantics and where failures are translated **13**

18 Testing and audit map **13**
18.1 Historical bit-for-bit regression 13
18.2 Exact-rational trusted-kernel tests 13
18.3 High-precision elementary tests 13
18.4 Representation and semantic tests 13
18.5 Prepared-session tests 14

19 How to modify AutoIA without breaking the proof architecture **14**
19.1 Adding a new scalar elementary function 14
19.2 Adding a new semantic numerical primitive 14
19.3 Adding a new representation 15
19.4 Adding a new range method 15
19.5 Changing the prepared-session cache 15
19.6 Adding a GPU backend 15

20 Proposal-versus-proof checklist for code review **15**

21 Compact source-map summary **16**

1 Repository architecture at version 0.21.0.dev1

The production package is organized into layers with deliberately restricted responsibilities. The principal directories are:

Production location	Responsibility
`autoia/api.py`	Stable public one-box/subdivision API, input validation, typed exception translation, and user-model guarding.
`autoia/session.py`	Prepared repeated-query sessions, certificate cache, rigorous certificate restriction, serial/thread/process many-box execution, and streaming APIs.
`autoia/trusted/`	Proof-critical outward-rounded binary64 interval arithmetic, runtime environment audit, rigorous reductions, and trusted scalar elementary-function kernels.
`autoia/polynomial/`	Representation-independent sparse polynomial algebra, monomial/range logic, Bernstein transforms, and rigorous coordinate recentering.
`autoia/enclosures/`	Scalar validated representations and representation-level arithmetic: interval, polynomial, shared-generator, piecewise, power, elementary lifting, and runtime contexts.
`autoia/semantic/`	Local NumPy/SciPy dispatch and semantic linear-solve certificates.
`autoia/controller/`	Core execution orchestration, final range policy, subdivision, split scoring, path-aware branching, and diagnostics.
`autoia/elementary/`	Generic Taylor coefficient/remainder machinery above the trusted scalar kernels.
`autoia/special.py`	Public error-function and normal PDF/CDF convenience functions.
`autoia/results.py`	Public result aliases/facade.
`autoia/policies.py`	Public range/subdivision/branch policy classes with typed validation.
`autoia/exceptions.py`	Frozen public exception taxonomy.

The low-level dependency direction is intentionally one-way. In particular, `autoia/trusted/` must not import controller, semantic, enclosure, or API layers. Boundary tests enforce this architectural rule.

2 One-box call path from public API to proof result

A normal user call is

```
result = autoia(model, parameter_intervals,
                order=4,
                representation="polynomial")
```

The production flow is:

1. `autoia/api.py`: `autoia(...)` validates public inputs and creates/validates the backend and policies.

2. `autoia/api.py`: `_guard_model(...)` wraps the user model so user exceptions and AutoIA proof failures are classified separately.

3. `autoia/controller/execution.py`: the controller constructs rigorous centered input domains and initial representation objects.

4. `autoia/semantic/dispatch.py`: the user's function executes in a private shadow namespace whose NumPy/math/SciPy references can route supported operations to validated semantics without mutating global modules.

5. Scalar operations are performed by `ValidatedValue` and its concrete representation classes under `autoia/enclosures/`.

6. Recognized `np.linalg.solve` calls are handled by `autoia/semantic/linear_solve.py`.

7. If a Python scalar predicate cannot be decided, path handling under `autoia/controller/branching.py` re-executes feasible paths.

8. Final representation ranges are chosen by `autoia/controller/ranging.py` and the polynomial range engines under `autoia/polynomial/ranging.py`.

9. `autoia/controller/execution.py` constructs an `AutoIAResult`; path-aware execution constructs an `AutoIAPathwiseResult`.

The important engineering point is that the user program is still executed as ordinary Python. AutoIA changes the objects and selected library references seen by that program, rather than translating the complete function into a separate symbolic language.

3 Public input intervals and rigorous centering

The mathematical input contract is

$$p_i \in [\underline{p}_i, \overline{p}_i].$$

The public interval class is exposed from `autoia/intervals.py`, which facades the trusted interval representation.

The centered-domain algorithm

$$q_i = p_i - p_{c,i} \in [-h_i, h_i]$$

is implemented by

`autoia/controller/execution.py`: `rigorous_center_and_halfwidth`.

The center is an ordinary finite binary64 proposal. The difference between the user interval and this center is computed through the trusted backend, and h_i is chosen from the resulting rigorous interval. This is where the known symmetric-domain boundary enlargement enters the current design.

Initial uncertain parameter objects are constructed in `autoia_single(...)` in the same module. Depending on the requested representation, it calls:

- `PolynomialEnclosure.parameter(...)`;
- `SharedGeneratorPolynomialEnclosure.parameter(...)`; or
- `IntervalEnclosure(...)`.

4 Representation-independent value dispatch

The abstract mathematical invariant

$$z(\mathbf{q}) \in \Gamma_E(\mathbf{q})$$

is represented in code by a `ValidatedValue` wrapper and a concrete representation object.

The representation-independent wrapper is in

`autoia/enclosures/value.py`.

It provides arithmetic operator hooks and delegates to a promoted concrete representation. The promotion priority is strategy, not proof. Soundness comes from each target representation's `convert_from(...)` implementation.

The currently important concrete scalar families are:

Mathematical family	Python class	Production file
$P(\mathbf{q}) + R$	`ParameterizedEnclosure` / `PolynomialEnclosure`	`autoia/enclosures/polynomial.py`
Single interval I	`IntervalEnclosure`	`autoia/enclosures/interval.py`
$P(\mathbf{q}) + G\boldsymbol{\varepsilon} + R$	`SharedGeneratorPolynomialEnclosure`	`autoia/enclosures/generator.py`
Finite path union	`PiecewiseEnclosure`	`autoia/enclosures/piecewise.py`

The generator representation advertises a higher promotion priority than the plain polynomial representation so shared provenance is retained when sound conversion is available.

5 Polynomial algebra: equations to functions

Sparse point-coefficient polynomial algebra lives in `autoia/polynomial/algebra.py`.

5.1 Multi-indices

The mathematical objects

$$\alpha = (\alpha_1, \ldots, \alpha_m), \qquad |\alpha| = \sum_i \alpha_i$$

are Python tuples of integers. Important helper functions include:

- `zero_alpha(m)`;
- `basis_alpha(m,i)`;
- `total_degree(alpha)`;
- `multi_indices_exact_degree(m,degree)`.

The exact-degree multi-index collections are cached because they are purely combinatorial and independent of numerical values.

5.2 Monomial magnitude

The proof quantity

$$H_\alpha \geq \max_{\mathbf{q}\in\mathcal{Q}} |\mathbf{q}^\alpha|$$

is computed by `monomial_magnitude_upper`. Its parity-aware interval version is `monomial_range_symmetric`. Backend-local caching avoids recomputing the same proof bound for repeated (α, h) combinations.

5.3 Addition

The rigorous point-coefficient addition rule is implemented by `rigorous_poly_add_point`. The function returns both the stored point polynomial proposal and an interval enclosing coefficient-rounding error after that error has been propagated through monomial ranges.

5.4 Truncated multiplication

The mathematical coefficient convolution

$$C_\gamma = \sum_{\alpha+\beta=\gamma} c_\alpha d_\beta$$

with a retained degree cutoff is implemented by `rigorous_poly_mul_truncated`. Sparse convolution structure is cached by `_polynomial_product_plan`. The numerical coefficients themselves are not cached. Every kept coefficient is stored as a point proposal with rigorous coefficient error, and discarded high-degree contributions are ranged into an interval remainder.

5.5 Reciprocal proposal

`build_reciprocal_predictor_point` constructs a point polynomial proposal for a reciprocal. The representation-level reciprocal certificate is implemented above this helper in `autoia/enclosures/polynomial.py`; it uses the residual identity rather than trusting the proposal.

6 Polynomial range methods in code

All principal polynomial range engines are in `autoia/polynomial/ranging.py`.

Function	Mathematical role
`range_termwise_scalar`	Sum rigorous monomial intervals.
`derivative_range_termwise`	Rigorous range of $\partial P/\partial q_i$.
`range_monotonicity_scalar`	Detect sign-definite derivatives and fix monotone coordinates at extrema.
`power_coeff_unit_box_interval`	Convert power coefficients toward the full Bernstein representation on a unit box.
`range_bernstein_scalar`	Rigorous Bernstein coefficient hull.
`power_coeff_reduced_unit_box_interval`	Bernstein conversion after monotone coordinates have been fixed.
`range_monotonicity_reduced_bernstein_scalar`	Combined structural range method used by the adaptive policy.

Function	Mathematical role
`range_polynomial`	Public/internal range-method selector and portfolio.

Final output policy is selected by `autoia/controller/ranging.py`. The controller layer decides which sound range result to request; the polynomial module performs the actual mathematical enclosure.

7 Trusted binary64 layer: the proof boundary

The trusted layer is the most important place to maintain architectural discipline. Its production files are:

- `autoia/trusted/environment.py`: runtime floating-point audit;
- `autoia/trusted/binary64.py`: scalar outward-rounded arithmetic;
- `autoia/trusted/interval.py`: trusted interval type and elementary interval operations;
- `autoia/trusted/reductions.py`: rigorous vector/matrix reductions;
- `autoia/trusted/elementary.py`: trusted scalar elementary and special functions.

The public backend facade is exposed as `RigorousBinary64Backend` from `autoia/backends.py`. The underlying implementation is imported from the trusted package.

7.1 Long dot products

The rigorous dot-product contract

$$\sum_i a_i b_i \in I$$

is implemented by `rigorous_dot_point`. It selects either the historical scalar reference implementation or the vectorized outward pairwise implementation according to the backend's `reduction_method` and term threshold.

Step 9b extends the same proof pattern to

- `rigorous_dot_point_interval`;
- `rigorous_dot_interval`;
- point/interval matrix-vector products;
- point/interval matrix-matrix products;
- nonnegative upper-bound reductions.

The batched matrix kernels are intentionally written as collections of explicit outward product intervals and pairwise interval sums. They are not wrappers around an unverified BLAS reduction.

8 Elementary functions: trusted scalar kernel plus generic Taylor lifting

The implementation is split deliberately.

Trusted scalar layer. `autoia/trusted/elementary.py` implements rigorous scalar/interval kernels for $\sqrt{\cdot}$, exp, log, sin, cos, erf, normal PDF/CDF, and constant powers. Important helpers include `pi_interval`, `ln2_interval`, and the point/interval variants of every supported function.

Generic polynomial lifting. `autoia/elementary/taylor.py` builds Taylor coefficient intervals and remainder radii. `taylor_coefficients` and `taylor_remainder_radius` contain the mathematical function-specific recurrences and tail bounds; `build_taylor_data` packages them for the enclosure layer.

Representation dispatch. `autoia/enclosures/elementary.py` applies the Taylor data to polynomial, interval, and generator-valued arguments. This division keeps transcendental proof facts out of user dispatch code.

9 Power semantics in code

Constant exponent handling is centralized in `autoia/enclosures/power.py` and trusted support in `autoia/trusted/elementary.py`. Validated arithmetic routes `x ** p`, `pow(x,p)`, and supported NumPy/math power calls to this layer. The trusted functions `real_power_point_interval` and `real_power_interval` enforce the real-domain restrictions described in the mathematical specification.

10 Local NumPy/math/SciPy semantic dispatch

Global monkeypatching is deliberately avoided. The key code is `autoia/semantic/dispatch.py`.

10.1 Prepared shadow function

`prepare_model_for_dispatch` and `prepare_dispatch_model` clone the user function with a private shadow global namespace. Same-module helper functions can also be cloned into that namespace. NumPy and `math` names are replaced by proxies that route only supported validated calls; the real modules remain unchanged.

The class `PreparedDispatchModel` is the static object reused by `PreparedAutoIA` in Step 9c.

10.2 NumPy proxy

`_NumpyProxy` and `_LinalgProxy` provide local dispatch for array constructors, elementary operations, norms, and `linalg.solve`. Object-array construction is used so ordinary NumPy indexing/broadcasting invokes validated scalar arithmetic.

10.3 SciPy compatibility

`autoia/semantic/scipy_dispatch.py` recognizes supported public SciPy surfaces such as `scipy.special.erf`, `ndtr`, legacy `scipy.stats.norm`, frozen normal distributions, and the newer `scipy.stats.Normal` API when present. SciPy syntax is a front-end only; the proof routes to AutoIA's own rigorous normal/error-function kernels.

11 Semantic linear solve: theory-to-code map

The main implementation is `autoia/semantic/linear_solve.py`. It contains both proposal builders and certificate routines. This file is intentionally larger than most modules because it holds several no-regression solve paths.

11.1 General polynomial predictor

The recurrence

$$A_0 x_\alpha = b_\alpha - \sum_{\substack{\beta \neq 0 \\ \beta \leq \alpha}} A_\beta x_{\alpha-\beta}$$

is implemented by `build_solution_polynomials_general`. Coefficients of the same total degree are solved together as a multiple-right-hand-side NumPy solve so one reference factorization can serve a whole degree. This is proposal-only performance work from Step 9.

11.2 Residual coefficient construction

`residual_coefficient_intervals_general` constructs rigorous interval coefficients for

$$r = b - AX.$$

`range_interval_coefficient_poly` and related functions rigorously range the resulting interval-coefficient polynomials.

11.3 Preconditioner portfolio

`polynomial_preconditioner_portfolio` generates center and matrix-family representatives. The output candidates are ordinary point matrices. Candidate quality is not trusted.

11.4 Defect and contraction proof

`preconditioned_defect_bounds_general` establishes a rigorous matrix bound for

$$|I - CA|.$$

`evaluate_general_polynomial_candidate` combines residual and defect bounds, checks $\rho < 1$, constructs the scalar/componentwise correction, and returns the certified candidate.

The older affine path remains separately implemented for historical bit-for-bit regression. `polynomial_semantic_solve_general` selects the general path only when required.

11.5 Generator-aware solve

The Step-7b path begins around `generator_semantic_solve`. Important functions include:

- `_generator_sensitivity_proposal`;
- `_generator_predictor_vector`;
- `_generator_residual_vector`;
- `_generator_defect_bounds`;
- `_evaluate_generator_solve_candidate`.

These functions retain incoming generator identifiers in the explicit solution predictor and prove the remaining error by the same contraction theorem over the joint physical/generator domain.

11.6 Dynamic solve context

`SolveContext` and `GenericSolveContext` provide the run-local semantic solve dispatcher. The local dispatch mechanism passes the current context explicitly/context-locally rather than replacing global `np.linalg.solve`.

12 Path-aware control flow in code

Path logic lives principally in `autoia/controller/branching.py` and supporting predicate objects in `autoia/enclosures/branching.py`.

The internal `_PathState` tracks the current physical box, forced Boolean outcomes already chosen during path enumeration, split depth, and trace information. `autoia_pathwise` is the path-aware execution controller.

For exact affine one-variable predicates, `_exact_single_variable_root` tries to recover a direct physical cut location. Otherwise `_predicate_split_direction` proposes a geometric split direction. `_make_piecewise_result` assembles completed feasible paths into the public piecewise result.

Internal branch-control signals derive from `BaseException` rather than ordinary `Exception`; this prevents a user's broad `except Exception:` from swallowing AutoIA's internal request to re-run a path under a forced predicate outcome.

13 Adaptive subdivision and cheap split scoring

Prescribed and adaptive subdivision are implemented by `autoia/controller/subdivision.py`. The practical cheap score is separated into `autoia/controller/split_scoring.py`.

The score's polynomial re-centering is intentionally a cheap floating prediction, not a proof. Functions such as `translate_polynomial_one_direction`, `derivative_range_termwise_float`, and `score_split_direction` estimate the structural effect of a hypothetical half split. `cheap_score_key` implements the lexicographic ordering.

After a direction is selected, `autoia/controller/subdivision.py` actually certifies the chosen two children and measures realized gain before accepting the split. The historical reference mode remains available through `split_method="lookahead"`.

14 Prepared sessions and certificate restriction in code

Step 9c is implemented primarily in `autoia/session.py` and `autoia/polynomial/recenter.py`.

14.1 Public preparation object

`prepare(model,...)` returns `PreparedAutoIA`. Its constructor:

1. validates the fixed model/configuration;
2. creates or accepts one `RigorousBinary64Backend`;
3. creates one `CertificateCache`;
4. wraps the user model through `_guard_model`; and
5. calls `prepare_dispatch_model` exactly once.

The session then exposes `certify`, `certify_many`, `certify_many_bounds`, `iter_certify_many`, and `iter_bounds_many`.

14.2 Exact cache

`CertificateCache` is a bounded LRU cache keyed by exact stored endpoint tuples. `get_exact` is an O(1) dictionary lookup. An exact cache hit returns a shallow result wrapper so per-request `cache_info` metadata does not mutate a result object previously returned to the user.

14.3 Containment cache

`get_smallest_container` scans the bounded cache for parent boxes that contain the query and chooses the smallest by log-volume. This choice is a heuristic among already sound candidate parents; no proof depends on it.

14.4 Restriction function

`_restrict_cached_result` performs the rigorous subbox restriction. For every new input interval it calls `rigorous_center_and_halfwidth`. For polynomial/generator representations it constructs the actual outward symmetric physical envelope and requires that envelope to lie inside the cached parent box.

The exact real center shift is enclosed by trusted subtraction:

```
backend.sub(point_interval(c_new), point_interval(c_old))
```

The resulting intervals are passed to `rigorous_recenter_point_polynomial` in `autoia/polynomial/recenter.py`.

That function implements the multivariate binomial coordinate transform. For every exact transformed coefficient it simultaneously computes:

- a rigorous interval using trusted operations and rigorous shift intervals;
- an ordinary binary64 point proposal using the stored shift proposal.

The difference is multiplied by a rigorous monomial range over the new half-width vector and accumulated in `extra_remainder`. The new representation is then final-ranged using the normal adaptive range policy.

14.5 Interval representation restriction

An interval-only cached result needs no polynomial coordinate transform. The same interval enclosure remains valid on any physical subbox. This can eliminate model execution but generally does not sharpen the output interval unless a fresh proof is performed.

15 Many-box CPU execution in code

`PreparedAutoIA.certify_many` normalizes a finite collection of boxes and supports three execution modes.

Serial. The session calls `certify` in input order and shares one cache/backend.

Thread. `ThreadPoolExecutor` runs chunks against the same prepared session. Locks protect session statistics/cache bookkeeping, while proof runtime state is context-local. The current Python-object workload is not guaranteed to benefit from threads.

Process. `ProcessPoolExecutor` with the `spawn` context uses `_process_initializer`. Each worker constructs one independent trusted backend and one `PreparedAutoIA` session, then reuses it for all assigned chunks. The model must be pickleable, normally meaning it is a module-level function.

`iter_certify_many` uses a bounded deque of pending futures so only a small number of chunks are in flight. This is the API intended for very large streams. `iter_bounds_many` discards full result objects after yielding copies of lower/upper arrays.

16 Result objects and mathematical diagnostics

The primary result dataclass is defined in `autoia/controller/execution.py` as `AutoIAResult`. It stores:

- flat `lower` and `upper` arrays;
- structured validated `objects`;
- backend name and `machine_rigorous` flag;
- semantic certificates;
- requested representation;
- range policy and final-range diagnostics;
- output shape.

Convenience properties reconstruct shaped outputs and expose generator IDs/matrix when present.

Path-aware results are implemented in `autoia/controller/branching.py` and exposed through `autoia/results.py`. They retain per-branch boxes, decisions, traces, and piecewise output enclosures in addition to global lower/upper hulls.

Prepared-session results receive a lightweight `cache_info` attribute identifying whether the query was a fresh certificate, an exact cache hit, or a restricted parent certificate.

17 Typed failure semantics and where failures are translated

Public exception classes are defined in `autoia/exceptions.py`. The important categories are:

- `InvalidInputError`;
- `UnsupportedOperationError`;
- `CertificationError`;
- `ResourceLimitError`;
- `NumericalLimitError`;
- `ModelExecutionError`;
- `InternalAutoIAError`.

`autoia/api.py`: `_translate_internal_exception` maps lower level failures to this taxonomy. The distinction between a failed proof and an unsupported operation is important: a contraction failure means the requested claim was not proved on that box, not that the model syntax is unsupported.

18 Testing and audit map

The production source is protected by several kinds of tests.

18.1 Historical bit-for-bit regression

The retired Version-0.11 reference implementation lives under `tests/support/oracle_v11.py`. Tests such as `tests/test_reference_regression.py`, `tests/test_semantic_solve_regression.py`, and trusted-kernel regression tests protect established arithmetic paths from accidental changes.

18.2 Exact-rational trusted-kernel tests

`tests/test_scalar_binary64_exact.py`, `tests/test_reductions_exact.py`, and Step-9/9b performance tests compare proof-critical binary64 enclosures to exact rational calculations.

18.3 High-precision elementary tests

Trusted transcendental functions are checked against independent high-precision references in `tests/test_trusted_elementary.py`, power tests, and the Step-3 audit tools.

18.4 Representation and semantic tests

Polynomial, generator, solve, branching, SciPy compatibility, array semantics, and controller tests exercise each representation and semantic boundary.

18.5 Prepared-session tests

`tests/test_prepared_manybox.py` verifies:

- one-shot versus prepared fresh equivalence;
- exact-cache model-execution avoidance;
- per-request cache metadata isolation;
- interior certificate restriction;
- fail-closed boundary reuse;
- generator-ID preservation through restriction;
- thread order preservation;
- compact bounds and streaming APIs;
- process-mode validation for non-pickleable functions.

The separate tool `tools/audit_manybox.py` measures repeated-query performance, checks sampled truth containment, and exercises process-mode outer parallelism.

19 How to modify AutoIA without breaking the proof architecture

This section gives implementation recipes tied directly to the mathematical contracts.

19.1 Adding a new scalar elementary function

A mathematically smooth scalar function should normally be added in four places:

1. add a trusted scalar/interval kernel or a trusted domain/range primitive in `autoia/trusted/elementary.py`;
2. add Taylor coefficient/remainder formulas in `autoia/elementary/taylor.py`;
3. connect representation-level lifting in `autoia/enclosures/elementary.py`;
4. add local NumPy/math/SciPy dispatch only for actual public syntax that users are expected to write.

The trusted layer must not call unverified platform transcendental functions as proof facts unless a separate rounding guarantee is established.

19.2 Adding a new semantic numerical primitive

For an operation such as a future nonlinear root solver or eigensystem:

1. define the exact mathematical semantics being certified;
2. permit arbitrary ordinary-floating proposal calculations;
3. derive an independent residual/fixed-point/inclusion certificate;

4. place the semantic proof under `autoia/semantic/`;

5. route recognized syntax through local dispatch without global monkeypatching;

6. add generic fallback behavior or a typed unsupported failure when the proof cannot apply.

The semantic primitive should not be accepted merely because an approximate library routine returned a plausible result.

19.3 Adding a new representation

A new representation must define a concretization and sound arithmetic/conversion rules. It should implement the representation contract expected by `ValidatedValue`, including a stable `representation_name`, range operation, arithmetic where supported, and inclusion-preserving `convert_from`. Promotion priority is chosen only after the conversions are proved sound.

19.4 Adding a new range method

Keep the proof implementation under `autoia/polynomial/` or an analogous low-level range module. Controller policy may decide when to use it, but the method itself should return an interval independently known to contain the exact represented range. A portfolio may then intersect it with other rigorous methods.

19.5 Changing the prepared-session cache

The public `PreparedAutoIA` API should not require a specific cache data structure. The current bounded LRU scan can later be replaced by a spatial index. A new index may alter parent-selection cost and reuse frequency, but must never skip the explicit proof that the query's actual represented centered domain lies inside the parent's certified domain before polynomial restriction.

19.6 Adding a GPU backend

The current Python object representation should not simply be copied to a GPU. The preferred architecture is a numeric structure-of-arrays backend behind the already frozen prepare/certify-many public API. GPU proposal kernels can be introduced first because their outputs can be revalidated by the existing CPU trusted proof layer. A future proof-producing GPU backend would require its own audited directed-rounding primitives and independent exact-reference tests.

20 Proposal-versus-proof checklist for code review

Before accepting a modification, ask:

1. What exact mathematical set or equation is being certified?

2. Which computed quantities are only proposals?

3. Where is the exact residual, interval extension, or domain condition checked?

4. Does any proof-critical calculation use ordinary round-to-nearest arithmetic without an enclosing error calculation?

5. If a representation loses structure, is the conversion explicitly inclusion-preserving?

6. If correlation is shared, what proves that two terms refer to the same auxiliary variable?
7. If a path is discarded, what proves it is infeasible?
8. If a cached parent certificate is reused, what proves the new represented domain is contained in the parent domain?
9. If performance batching changes arithmetic order, is every batched reduction still a composition of rigorous interval operations?
10. Is there an exact/high-precision/regression test that would fail if the new proof boundary were accidentally weakened?

21 Compact source-map summary

Mathematical idea	Main implementation location
Input centering $p \mapsto q$	`autoia/controller/execution.py`
Outward scalar interval arithmetic	`autoia/trusted/binary64.py`, `autoia/trusted/interval.py`
Rigorous dot/matrix reductions	`autoia/trusted/reductions.py`
Polynomial algebra and truncation	`autoia/polynomial/algebra.py`
Termwise/monotonicity/Bernstein range	`autoia/polynomial/ranging.py`
Polynomial coordinate restriction/recentering	`autoia/polynomial/recenter.py`
$P + R$ representation	`autoia/enclosures/polynomial.py`
$P + G\varepsilon + R$ representation	`autoia/enclosures/generator.py`
Piecewise representation	`autoia/enclosures/piecewise.py`
Taylor lifting	`autoia/elementary/taylor.py`, `autoia/enclosures/elementary.py`
Trusted transcendentals	`autoia/trusted/elementary.py`
Semantic linear solve	`autoia/semantic/linear_solve.py`
NumPy/math dispatch	`autoia/semantic/dispatch.py`
SciPy normal/error-function syntax	`autoia/semantic/scipy_dispatch.py`
Path-aware execution	`autoia/controller/branching.py`
Adaptive split score	`autoia/controller/split_scoring.py`
Subdivision orchestration	`autoia/controller/subdivision.py`
Prepared/many-box execution	`autoia/session.py`
Public error taxonomy	`autoia/exceptions.py`, translated in `autoia/api.py`

The main design principle behind this map is that the mathematical proof should remain visible in the source architecture. Proposal code may evolve rapidly for performance, but every accepted certificate should still be traceable to a small number of explicitly audited enclosure operations and documented mathematical identities.